

Plano de Refatoração do Backend em 3 Meses

Projeto: Fiestou

Responsável técnico: Pedro (Tech Lead / Desenvolvedor Principal)

Este plano foi reduzido para execução direta: foco no que realmente decide sucesso da migração sem travar a operação atual da Fiestou.

objetivos claros

riscos controlados

migração por ondas

paridade funcional

Sumário curto

- 1. Objetivos e escopo real
- 2. Riscos e mitigação
- 3. Plano de migração em 12 semanas
- 4. Modelo de dados aproximado (RavenDB)
- 5. Go/No-Go, rollback e operação
- 6. Estrutura de API e solução .NET (final)

1) Objetivos e escopo real

Objetivo principal: migrar o backend para .NET + RavenDB sem perder estabilidade de receita, mantendo os fluxos críticos do produto funcionando durante todo o processo.

1.1 Objetivos de negócio

- Garantir continuidade de checkout, pedido e pagamento sem regressão crítica.
- Permitir evolução mais rápida de painéis (admin, lojista e cliente) com backend mais previsível.
- Reduzir gargalos operacionais em reconciliação financeira, jobs e monitoramento.

1.2 Escopo obrigatório (entra no ciclo)

Domínio	O que precisa estar pronto no remake	Critério de paridade
Catálogo	loja, produto, variações, adicionais, mídia e disponibilidade	mesmo comportamento de listagem, detalhe e edição
Checkout e pedido	carrinho, validações, frete, fechamento e split por loja	totais e regras iguais ao legado
Pagamento/financeiro	confirmação, fallback de conciliação, painel financeiro lojista/admin	status e valores consistentes em todo fluxo
Painéis	admin, lojista e cliente com autorização correta	sem vazamento de dados entre perfis/lojas

1.3 Escopo fora deste ciclo

- Reescrita completa do frontend.
- Novas integrações externas não críticas para cutover.
- Mudança de regra de negócio sem ganho direto para estabilidade/migração.

1.4 Entregáveis esperados ao final dos 3 meses

Entrega	Estado esperado	Observação prática
Nova API .NET em produção parcial ou total	fluxos críticos roteados com segurança	não precisa 100% do legado refeito, precisa cobrir o que sustenta operação e receita
RavenDB operando como banco principal dos domínios migrados	leitura e escrita estáveis	com rotina de backup, restore testado e observabilidade mínima
Pedidos e pagamentos com paridade validada	sem divergência crítica	principalmente totais, status, split por loja e histórico operacional
Rollback e cutover documentados	processo executável sem imprevisto	com checklist de decisão e evidências mínimas para avançar ou voltar

Regra de condução: paridade funcional vem antes de otimização. Primeiro reproduzir com segurança, depois acelerar.

2) Riscos e mitigação

Risco	Impacto	Mitigação objetiva
Divergência financeira entre legado e novo	alto	dupla verificação por pedido, reconciliação automática e fallback diário
Quebra em checkout/pedido	alto	canary por rota + rollback em minutos + smoke obrigatório em staging
Vazamento de dados entre lojas/perfis	alto	testes de autorização por perfil e escopo de loja em toda release
Inconsistência de migração de dados	médio/alto	backfill idempotente com checksum e relatório de paridade
Dependência externa instável (gateway)	médio/alto	retentativa com política clara + monitoramento + fila de reconciliação

2.1 Dependências externas críticas

- Validação final de modelagem RavenDB com Lucas (1 revisão de arquitetura).
- Validação final de desenho .NET com Eduardo (1 revisão de implementação e pipeline).

2.2 Gatilhos que bloqueiam o avanço

Sinal	Leitura prática	Ação imediata
Divergência em valor de pedido ou pagamento	o domínio ainda não está pronto para assumir ownership	congelar avanços da onda, corrigir cálculo e revalidar paridade
Erro de autorização entre perfis ou lojas	risco direto de exposição de dados	rollback imediato da rota ou flag correspondente
Backfill sem consistência repetível	migração ainda não está idempotente	não seguir para cutover até haver relatório confiável de paridade
Dependência externa sem fallback operacional	o domínio pode parar por fator fora do controle da aplicação	implementar fila, retentativa ou modo de reconciliação antes de avançar

3) Plano de migração em 12 semanas

Execução por ondas, mantendo legado ativo e roteando gradualmente tráfego para o novo backend.

Semana	Meta principal	Entrega mínima
1-2	fundação	solução .NET base, observabilidade, auth base, healthchecks e CI
3-4	catálogo	read/write de lojas e produtos + backfill inicial + paridade de listagem
5-6	checkout e pedidos	cálculo de totais, regras de entrega e criação de pedido com split correto
7-8	pagamento e reconciliação	status de pagamento, fallback de webhook e reconciliação contínua
9-10	painéis e hardening	painéis críticos em .NET, testes de autorização e performance p95
11-12	canary e cutover	go/no-go final, mudança progressiva, rollback validado e encerramento

3.1 Estratégia de migração de dados (MySQL → RavenDB)

1. **Fase A - Snapshot inicial:** exportar dados base de catálogo, usuários, lojas e pedidos ativos.
2. **Fase B - Backfill incremental:** aplicar deltas por janela, idempotente, com registro de progresso.
3. **Fase C - Cutover controlado:** definir owner por domínio e congelar escrita no legado só para o domínio transferido.

3.2 Ordem de migração por domínio

Ordem	Domínio	Por que entra nessa fase
1	Catálogo	tem alto volume de leitura e menor risco financeiro direto, servindo como base do restante
2	Checkout e pedidos	depende do catálogo pronto e concentra o contrato principal do produto
3	Pagamentos e financeiro	entra depois que pedido estiver consistente para não misturar duas fontes instáveis ao mesmo tempo
4	Painéis e operação	fecha o ciclo com as visões de admin, lojista e cliente já em cima da nova base

3.3 Gates entre ondas

- Nenhuma onda avança sem smoke funcional aprovado em staging.
- Nenhuma escrita troca de dono sem rotina de rollback testada.
- Nenhum fluxo financeiro assume ownership novo sem evidência de reconciliação consistente.
- Toda troca de roteamento precisa poder voltar via flag, sem deploy emergencial complexo.

Critério simples de qualidade de migração: sem divergência crítica em valores de pedido/pagamento e sem quebra de autorização por loja.

4) Modelo de dados aproximado (RavenDB)

Modelo inicial por agregados de negócio da Fiestou. Ajustes finos entram depois da validação pontual dos mentores.

Documento	Campos principais	Observações de uso
stores/{id}	identidade da loja, status, localização, regras	base para catálogo, pedido e escopo de autorização
products/{id}	dados do item, tipo comercial, variações, adicionais, mídia	materializa leitura rápida para listagem e página de produto
orders/{id}	cliente, itens, split por loja, totais, status, timeline	documento central para painel admin/lojista/cliente
payments/{id}	transação, método, status, eventos, reconciliação	separado de pedido para auditoria e recuperação de estado
seller_finance/{storeId}	saldo, recebimentos, antecipações, pendências	fonte do painel financeiro do lojista

4.1 Índices iniciais recomendados

- Produtos por loja + status + data de atualização.
- Pedidos por loja + status + data de criação.
- Pagamentos por status + referência do pedido.
- Financeiro por loja + competência (dia/mês).

4.2 Decisões práticas de modelagem

Ponto	Direção proposta	Motivo
Produto	documento mais completo, com dados suficientes para leitura direta em página pública e painel	reduz joins mentais e custo de consulta em telas muito acessadas
Pedido	manter snapshot dos itens, valores e dados operacionais relevantes	pedido não pode depender de mudanças futuras do cadastro do produto
Pagamento	separar em documento próprio, ligado ao pedido por referência	facilita auditoria, webhook, reconciliação e correções sem corromper o pedido
Financeiro do lojista	visão agregada por loja, com eventos ou lançamentos derivados do fluxo principal	permite painel rápido sem recálculo pesado a cada acesso

5) Go/No-Go, rollback e operação

5.1 Critérios de Go para avançar cada onda

- Paridade funcional comprovada nos fluxos críticos da onda.
- Sem incidente crítico aberto em produção relacionado à onda.
- Métricas mínimas de latência e erro dentro do baseline aceitável.
- Plano de rollback validado em staging e documentado.

5.2 Critérios de rollback imediato

- quebra de checkout ou criação de pedido;
- divergência financeira crítica confirmada;
- erro de autorização com risco de dados de outra loja;
- taxa de erro acima do limite acordado por janela contínua.

5.3 Operação mínima obrigatória

Frente	Obrigatório
Observabilidade	logs estruturados, tracing por request e métricas de negócio
Backup	rotina automática de backup com restore validado em staging
Jobs	scheduler ativo para reconciliação e monitoramento operacional
Deploy	staging obrigatório antes de produção + checklist de liberação

5.4 Métricas mínimas após o cutover

- taxa de erro das rotas críticas dentro da faixa aceitável definida no baseline;
- tempo de resposta p95 sem degradação relevante nas telas e fluxos mais usados;
- nenhuma divergência financeira crítica aberta por mais de um ciclo operacional;
- nenhum incidente de autorização entre cliente, lojista e admin.

6) Estrutura de API e solução .NET (fica no final)

Estrutura proposta para organizar código por responsabilidade e manter manutenção simples no médio prazo.

6.1 Estrutura macro

```
src/  
  Fiestou.sln  
  Directory.Build.props  
  Directory.Packages.props  
  global.json  
  shared/  
    Fiestou.BuildingBlocks/  
  services/  
    Fiestou.Api/  
    Fiestou.Application/  
    Fiestou.Domain/  
    Fiestou.Infrastructure/  
    Fiestou.Contracts/  
    Fiestou.Jobs/  
  tests/  
    Fiestou.UnitTests/  
    Fiestou.IntegrationTests/  
    Fiestou.ContractTests/  
    Fiestou.ArchitectureTests/  
    Fiestou.PerformanceTests/  
  ops/  
    scripts/  
    docker/  
    observability/  
  docs/  
    adr/  
    runbooks/
```

6.2 O que entra em cada projeto

Projeto	Conteúdo principal	Não deve conter
Fiestou.Api	endpoints, auth/policies, middleware, OpenAPI, healthchecks	regra de negócio de domínio
Fiestou.Application	casos de uso, commands/queries, validação, autorização	detalhe de banco/gateway
Fiestou.Domain	entidades, value objects e políticas de negócio	dependência de framework/infra
Fiestou.Infrastructure	RavenDB, integrações, mensageria, cache e telemetria	decisão de fluxo de negócio
Fiestou.Contracts	DTOs versionados e contratos de eventos	lógica de aplicação
Fiestou.Jobs	workers, schedules, lock distribuído e retentativa	regra duplicada fora da Application

6.3 Fluxo padrão de requisição

O fluxo esperado é simples: a requisição entra pela API, passa por autenticação e validação, chama um caso de uso na Application, executa regra no Domain e delega persistência ou integração para Infrastructure. Jobs reutilizam os mesmos casos de uso sempre que possível, sem criar regra paralela.

6.4 Estratégia de testes (prioridade)

- **Obrigatório desde o início:** Unit, Integration e Contract tests.
- **Entrada gradual:** Architecture e Performance tests, sem bloquear o início.

Fechamento

O plano fecha com uma linha clara de execução: migrar por ondas, proteger os fluxos críticos da Fiestou e manter rollback simples em cada avanço relevante.

Documento interno da Fiestou para planejamento técnico e execução do remake de backend.